Distributed Messaging System - Backend Documentation

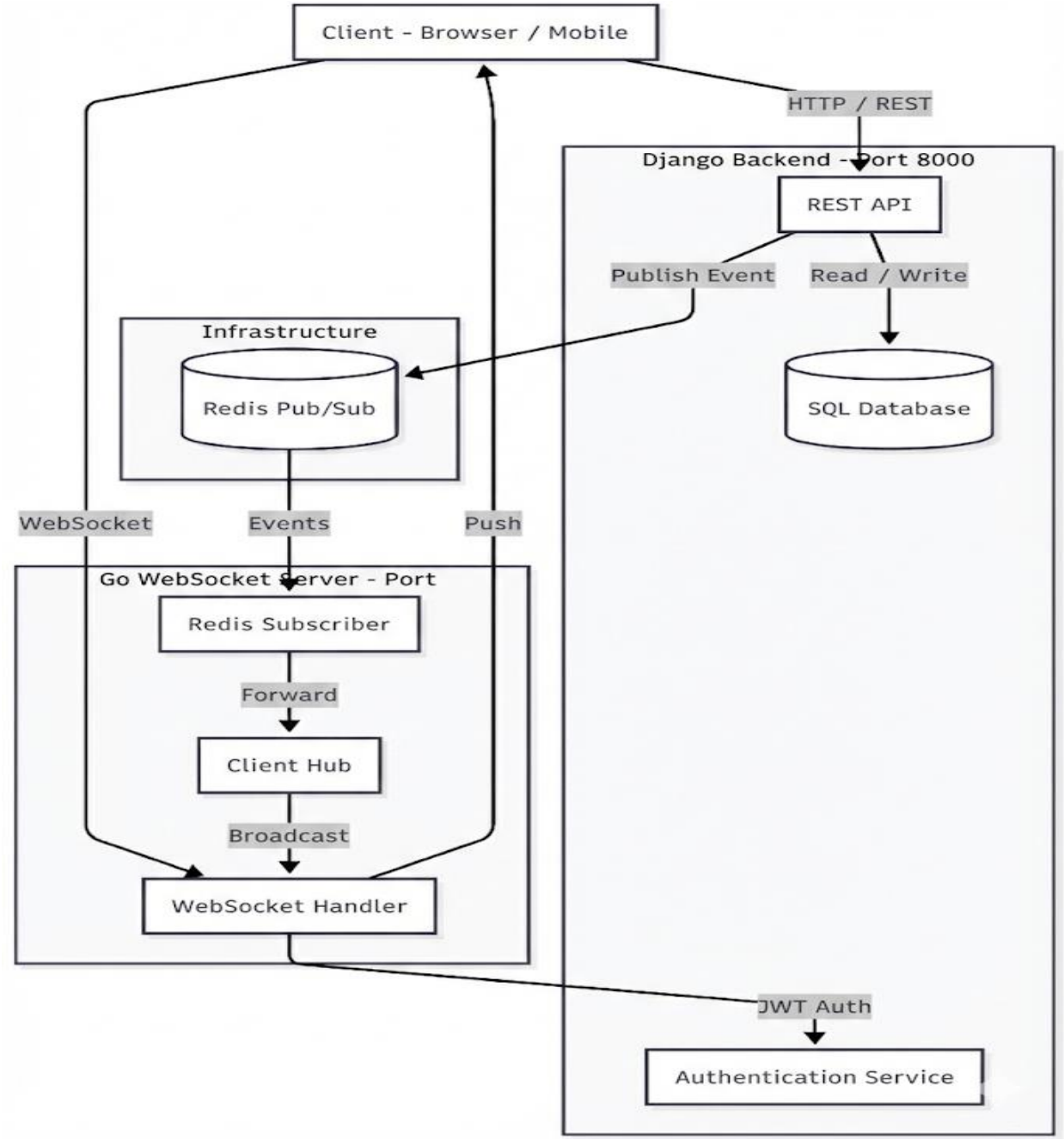
# 1. System Overview & Architecture

This document provides a deep technical overview of the Distributed Messaging System backend. The system utilizes a hybrid architecture combining **Django** for RESTful APIs and business logic with a **Go** WebSocket server for high-performance real-time communication.

## 1.1 Core Components

The system follows a microservices-like hybrid approach:

- **Django Backend (Port 8000):** Serves as the primary source of truth, handling user authentication, database interactions (SQLite/PostgreSQL), and REST API endpoints. It publishes real-time events to Redis.
- **Go WebSocket Server (Port 8080):** Handles persistent WebSocket connections. It subscribes to Redis channels to receive events from Django and broadcasts them to connected clients.
- **Redis:** Acts as the message broker (Pub/Sub) bridging the Django application and the Go WebSocket server.
- **Frontend:** Interacts with Django via HTTP (REST) and the Go server via WebSocket.




1.2 Architecture Data Flow

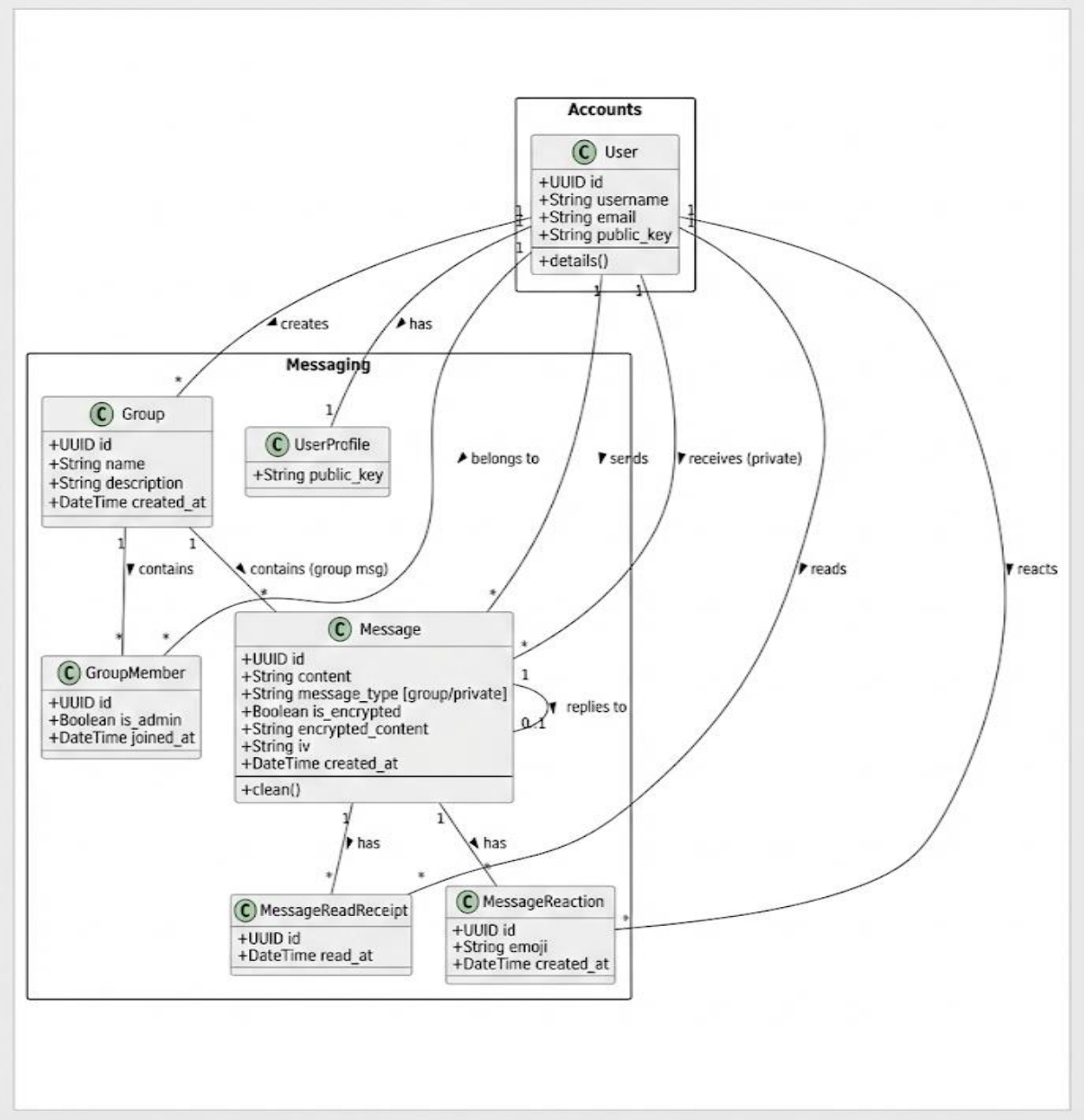
1. **Auth:** User authenticates with Django, receiving a JWT.
2. **Connection:** User connects to Go WebSocket server using the JWT.
3. **Action:** User performs an action (e.g., Send Message) via Django REST API.
4. **Persistence:** Django saves data to the database.
5. **Event:** Django publishes an event to Redis (e.g., group_message).
6. **Broadcast:** Go server receives the event and pushes it to relevant WebSocket clients.

2. Technology Stack

| Component | Technology | Details |
|---|---|---|
| **Languages** | Python 3.x, Go 1.20+ | Backend logic and Real-time services |
| **Frameworks** | Django & DRF, Gorilla WebSocket | Web framework and WebSocket implementation |
| **Database** | SQLite (Dev) / PostgreSQL | Relational data storage |
| **Message Broker** | Redis | Pub/Sub messaging |
| **Authentication** | JWT (SimpleJWT) | Stateless authentication |
| **Docs** | Swagger, PlantUML | API and System documentation |

3. Database Schema & Data Models

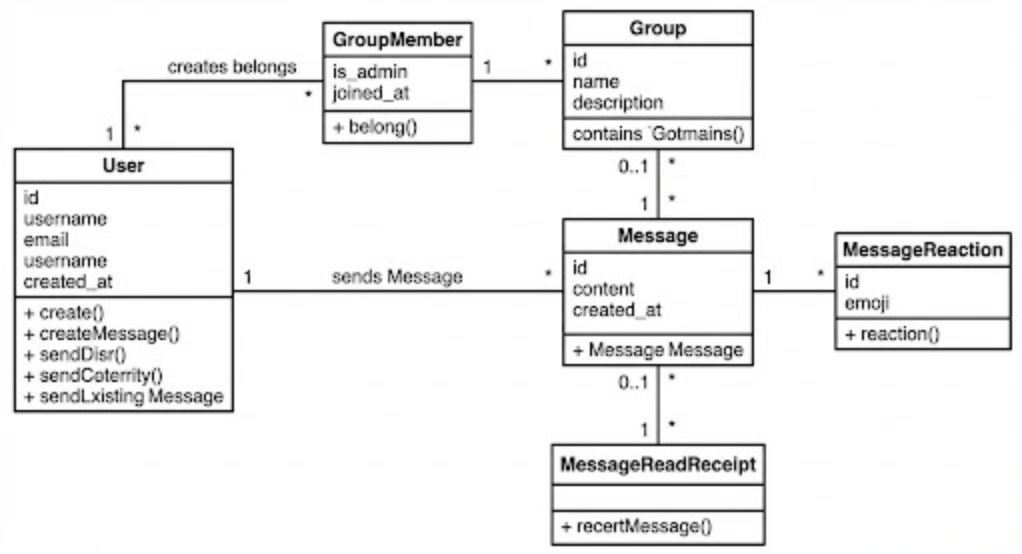
The core data models are defined in the Django backend. The system tracks Users, Groups, Messages, and Reactions.




3.1 Entity Relationships

The diagram below illustrates the relationships between Users, Groups, Messages, and Reactions. Key relationships include:

- Users can belong to multiple Groups.
- Messages belong to a Group or a Private Conversation.
- Messages can have multiple Read Receipts and Reactions.

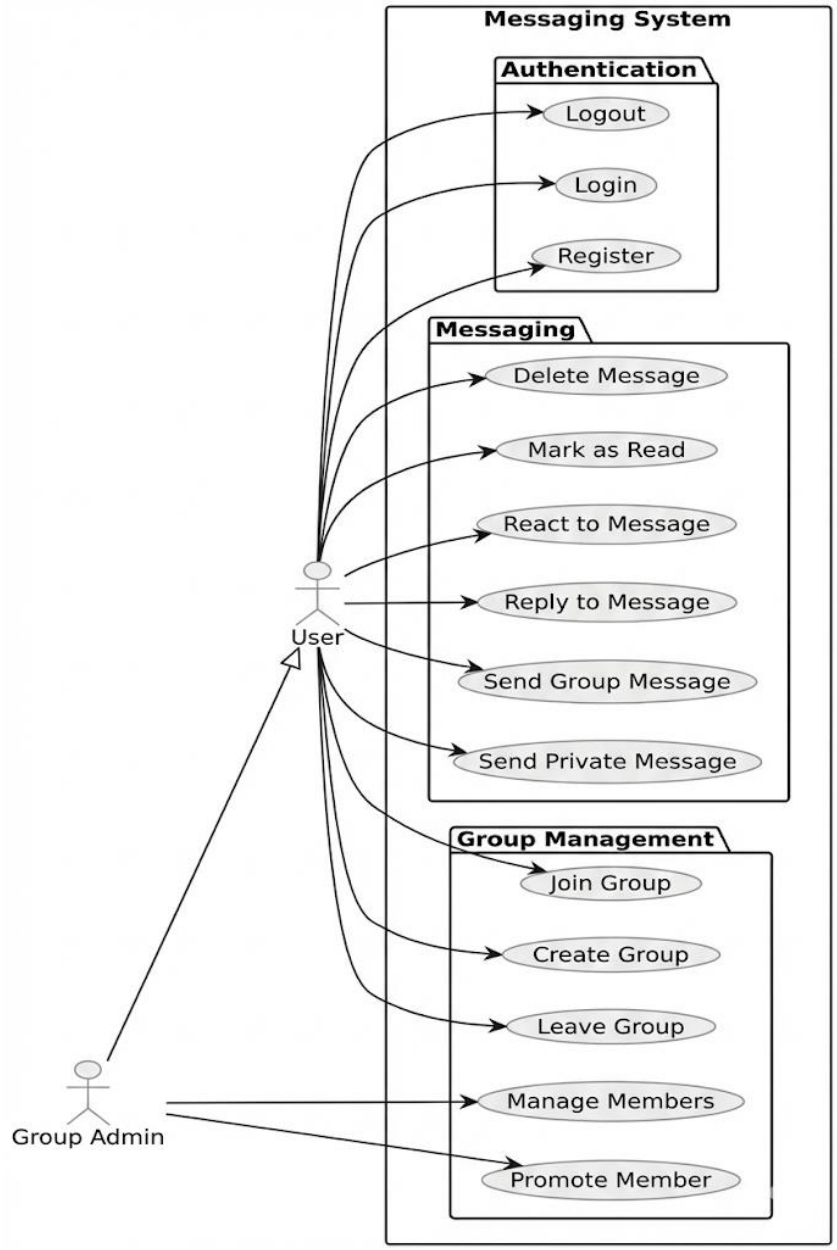



4. System Capabilities (Use Cases)

The system supports granular roles (User, Admin) and various messaging capabilities including encryption and reactions.

4.1 Core Features

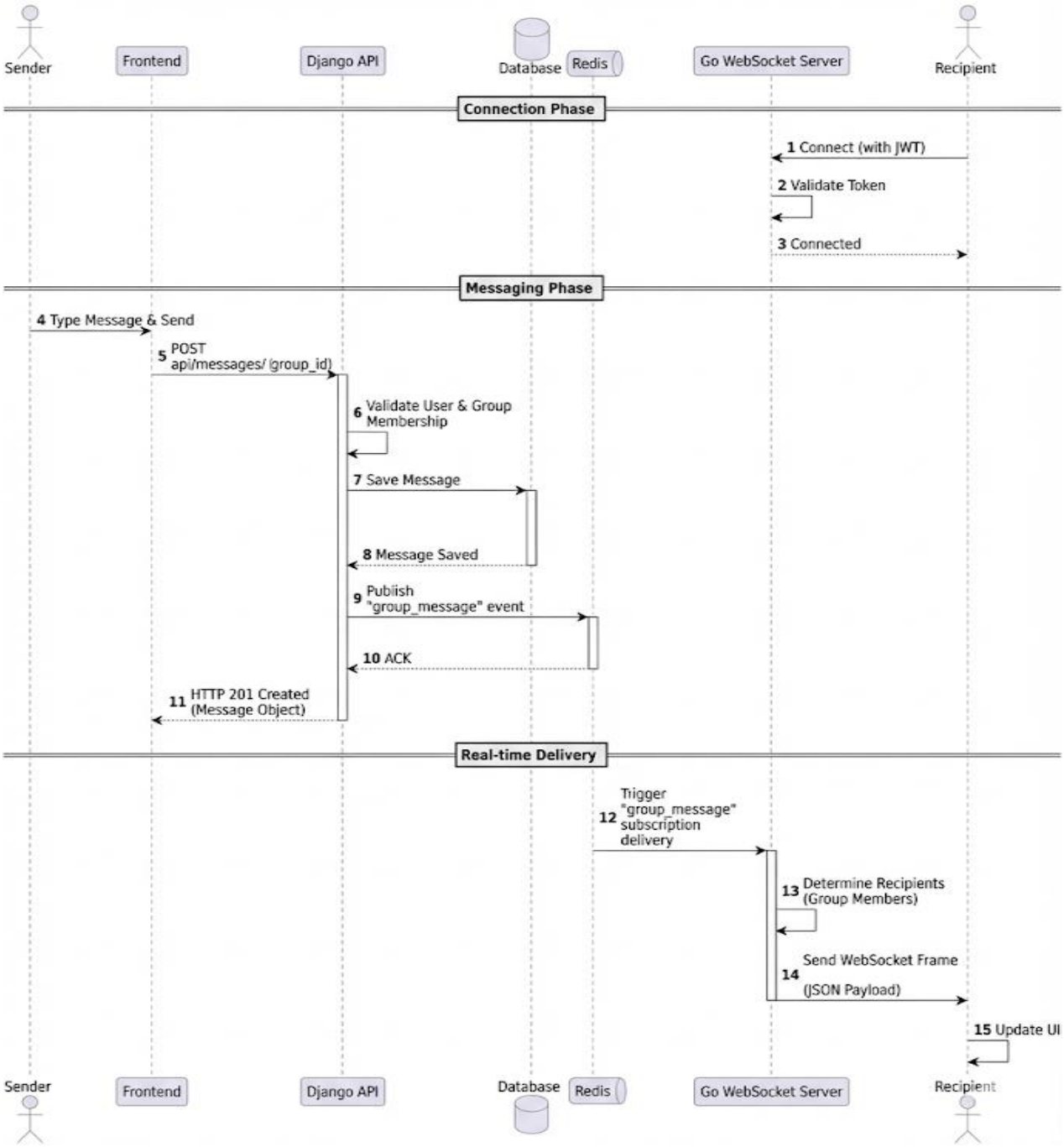
- **Authentication:** Register, Login, Logout.
- **Group Management:** Create, Join, Leave, Promote Members.
- **Messaging:** Private/Group messages, Reply, React, Mark as Read.




5. Sequence Diagrams

5.1 Real-time Group Message Flow

This scenario depicts the lifecycle of a message from the moment a user sends it until it is received by other group members in real-time.




5.2 User Join & Notification Flow

This scenario illustrates how the system handles a new user joining a group and notifying existing members.

- **User Action:** A user sends a POST request to the Django API to join a group.

- **API Processing:** Django creates a GroupMember entry in the database and publishes a "user_joined" event to Redis.
- **Broadcast:** The Go WebSocket server receives the event from Redis and broadcasts a "User X joined Group Y" message to all connected members of that group.

6. API Reference Overview

6.1 Accounts

| Method | Endpoint | Description |
|---|---|---|
| POST | /api/auth/login/ | Obtain JWT access/refresh tokens |
| POST | /api/auth/register/ | Register a new user account |
| GET | /api/users/me/ | Get current user profile |

6.2 Messaging

| Method | Endpoint | Description |
|---|---|---|
| GET | /api/groups/ | List all groups |
| POST | /api/groups/ | Create a new group |
| POST | /api/groups/{id}/join/ | Join a specific group |
| POST | /api/groups/{id}/leave/ | Leave a group |
| GET | /api/messages/?group={id} | Get distinct messages for a group |
| GET | /api/messages/?recipient={id} | Get private conversation history |
| POST | /api/messages/ | Send a new message (Group or Private) |

| | | |
|---|---|---|
| POST | /api/messages/mark_read/ | Batch mark messages as read |
| POST | /api/messages/{id}/react/ | Add/Toggle an emoji reaction |

7. Real-time Event Types

The Go server handles the following event types broadcasted via Redis:

- **group_message**: New message in a group.
- **private_message**: New direct message.
- **user_joined**: A user joined a group.
- **user_left**: A user left a group.
- **user_removed**: A user was kicked from a group.
- **member_promoted**: A member was promoted to admin.
- **message_deleted**: A message was deleted.
- **message_read**: A read receipt was generated.

8. Setup Instructions

To get the Distributed Messaging System up and running locally, follow these steps.

Prerequisites

- Python 3.10+ (for Django)
- Go 1.21+ (for WebSocket server)
- Redis 6.0+ (Message Broker)
- Node.js/npm (Frontend serving)

8.1. Django Backend Setup

1. **Navigate to the project root:** cd distributed-messaging
2. **Create and activate a virtual environment:**

```
Bash
python -m venv venv
source venv/bin/activate
```

3. **Install Python dependencies:** pip install -r requirements.txt
4. **Configure Environment:** Copy .env.example to .env. Ensure REDIS_URL matches your local Redis instance.
5. **Run Migrations:** python manage.py migrate
6. **Run the Server:** python manage.py runserver 8000

8.2. Go WebSocket Server Setup

1. **Navigate to the WebSocket directory:** cd websocket-server
2. **Install Go dependencies:** go mod download
3. **Configure Environment:** Copy .env.example to .env. **CRITICAL:** Ensure JWT_SECRET matches the Django settings.
4. **Run the Server:** go run main.go (Starts on port 8001).

8.3. Redis

Ensure Redis is running locally: redis-server

9. System Report

9.1. Approach: Hybrid Architecture

This project demonstrates a rigorous "Right Tool for the Job" approach by combining two powerful backend technologies:

- **Django (Python):** Used for the **Control Plane**. Django excels at rapid development of complex data models, business logic, authentication, and REST APIs. It serves as the "Source of Truth" for the system.
- **Go (Golang):** Used for the **Data Plane (Real-time)**. Go is chosen for its superior concurrency model (goroutines) and low-latency performance, making it ideal for maintaining thousands of persistent WebSocket connections.

9.2. Distributed Concepts Demonstrated

1. **Event-Driven Architecture:** The Django backend "fires and forgets" events to Redis, not caring who is listening.
2. **Decoupling:** The REST API and WebSocket server are completely decoupled.
3. **Pub/Sub Pattern:** Used for 1-to-Many broadcasting. A single message sent to the API is published once to Redis, then fanned out to multiple WebSocket clients.

9.3. Limitations and Future Improvements

- **Redis Cluster:** Implement Redis Sentinel or Cluster to remove the Single Point of Failure (SPOF).
- **Reliable Delivery:** Switch from Redis Pub/Sub to **Redis Streams** or **Kafka** to ensure at-least-once delivery of events.
- **Horizontal Scaling:** Modify the Go architecture to handle multi-node deployments using a Redis backplane strategy.